@mellaithy

SPRING BOOT
COMPLETE NOTES

(PART 1)

CORE & WEB LAYER

- CORE CONCEPTS
- ARCHITECTURE
- IoC & DI
- ANNOTATIONS
- CONFIGURATION
- REST APIs

WRITE SPRING BOOT SERVICES
THE WAY SENIORS ACTUALLY WRITE THEM

What is Spring Boot?

*Spring Boot is an **opinionated layer on top of the Spring Framework**. It does not replace Spring — it removes the setup ceremony so you can go from an empty folder to a running, production-shaped service in minutes.*

1 The one-line definition

*An open-source Java framework that builds on Spring to produce **stand-alone, production-grade** applications with minimal configuration.*

You run it as a plain `java -jar app.jar`. No application server to install, no XML to hand-write, no dependency version matrix to maintain yourself.

The distinction that matters

*Spring gives you the **programming model** (IoC, DI, AOP, MVC). Boot gives you the **delivery model** — defaults, packaging, and runtime wiring.*

2 The four pillars

- **Auto-configuration**
- **Starter dependencies**
- **Embedded server**
- **Production-ready features**

Everything else Boot does is a consequence of these four.

3 What each pillar actually does

Pillar	What it removes	How
Auto-config	Manual bean definitions	Conditional beans based on classpath
Starters	Version conflicts	Curated, tested dependency sets
Embedded server	External deployment	Tomcat / Jetty / Undertow in the jar
Actuator	Custom health plumbing	Ready-made ops endpoints

4 Where it fits in practice

REST & GraphQL APIs · Microservices · Payment & fintech services · Batch & scheduled jobs · Event-driven consumers · Mobile backends · Cloud-native workloads · Internal platform tools

Interview answer

"Boot reduces the time between deciding to build a service and having one that runs. It does that with sensible defaults you can always override — nothing is hidden, just pre-decided."

5 What it is not

- **Not** a replacement for Spring Framework.
- **Not** a code generator — nothing is scaffolded into your source tree.
- **Not** an application server.
- **Not** magic — every auto-configuration is a conditional `@Bean` you can inspect.

Prove it to yourself

Run with `--debug` and Boot prints a full auto-configuration report: what matched, what did not, and why.

6 From empty folder to running service

Initializr → Add starters → Write a `@RestController` → `mvn package` → `java -jar` → Serving traffic

7 Vocabulary you will hear

Term	Means
Bean	An object whose lifecycle the container owns
Context	The container instance holding all beans
Starter	A dependency bundle with curated versions
Auto-configuration	Conditional beans registered from the classpath
Fat jar	One executable jar with every dependency inside

Where the versions come from

*Boot 3.x requires **Java 17 or newer** and runs on the `jakarta.*` namespace, not `javax.*`. Anything written for Boot 2.x needs both changes before it will compile.*

Spring Framework vs Spring Boot

*Both are alive and both are used. The question is never "which is better" — it is **how much of the wiring do you want to own**.*

1 What plain Spring costs you

- Dependency versions chosen and aligned by hand.
- Component scanning, view resolvers, data sources declared explicitly.
- A servlet container installed, configured and deployed to separately.
- Health, metrics and externalised config built yourself.

Fair framing

None of this is a flaw. It is explicit control. Boot trades some of that control for speed — and lets you take it back one property at a time.

2 What Boot decides for you

- Curated dependency versions via the parent POM / BOM.
- Beans auto-registered based on what is on the classpath.
- An embedded server started by the application itself.
- Actuator, externalised config and profiles available immediately.

Convention over configuration

Boot picks a reasonable default for every decision. You only write configuration where your answer differs from the default.

3 Side by side

Concern	Spring Framework	Spring Boot
Configuration	Explicit XML or Java config	Auto-configuration, overridable
Dependencies	Managed manually	Starters with curated versions
Server	External (deploy a WAR)	Embedded (run an executable jar)
Time to first run	Hours	Minutes
Observability	Assembled yourself	Actuator out of the box
Config per environment	Custom mechanism	Profiles + property precedence
Best suited to	Legacy estates, bespoke architecture	Services, APIs, cloud-native apps

4 Choosing, honestly

Reach for Boot for anything greenfield: services, APIs, containers, anything that will be deployed more than a handful of times.

Stay on plain Spring when you are inside an existing estate that already owns its wiring, or when a hard requirement conflicts with an auto-configuration you would end up disabling anyway.

Interview answer

"Boot is Spring with the assembly already done. Same core container, same beans, same lifecycle — the difference is who writes the configuration."

5 Taking control back

Auto-configuration is not all-or-nothing:

```
@SpringBootApplication(
  exclude = {
    DataSourceAutoConfiguration.class
  })
public class App { }
```

Rule of thumb

Define a bean yourself and Boot backs off — most auto-configurations are `@ConditionalOnMissingBean`.

Spring Boot Architecture

*A request enters at the web layer, business rules run in the middle, persistence sits at the edge. Keeping these **three responsibilities separate** is most of what "good Spring Boot code" means.*

1 The layers

Presentation — @RestController

↓

Business — @Service

↓

Data access — @Repository

↓

Database

The dependency rule

Each layer knows only the one below it. A repository never imports a controller; a controller never builds a query.

2 Auto-configuration, concretely

Boot inspects the classpath and registers beans conditionally:

```
// present? -> configure it
@ConditionalOnClass
@ConditionalOnMissingBean
@ConditionalOnProperty
```

Read it yourself

The conditions live in `spring-boot-autoconfigure`. They are ordinary classes — open them.

3 Starter dependencies

A starter is a POM with no code — it exists purely to pull in a tested set of libraries.

Starter	Brings in
spring-boot-starter-web	Spring MVC, Jackson, embedded Tomcat
spring-boot-starter-data-jpa	Spring Data JPA, Hibernate, JDBC pool
spring-boot-starter-security	Spring Security core + web filter chain
spring-boot-starter-validation	Jakarta Bean Validation + Hibernate Validator
spring-boot-starter-test	JUnit 5, Mockito, AssertJ, Spring Test

4 Embedded server

- **Tomcat** — the default in `starter-web`.
- **Jetty / Undertow** — swap by excluding Tomcat.
- **Netty** — used instead when you choose WebFlux.

```
<!-- swap Tomcat for Undertow -->
<exclusions>
  <exclusion>
    <artifactId>spring-boot-starter-tomcat
    </artifactId>
  </exclusion>
</exclusions>
```

5 Startup sequence

SpringApplication .run() → **Prepare Environment** → **Create ApplicationContext** → **Run auto-config** → **Instantiate beans** → **Start embedded server** → **Ready**

The `Environment` is built first — which is why properties and profiles are already resolved by the time any bean is created.

Common misconception

The embedded server starts near the end. If a bean fails to construct, the port never opens — that is deliberate fail-fast behaviour.

Interview answer

"`SpringApplication` prepares the environment, creates the context, applies auto-configuration, instantiates singletons, then starts the server. Fail-fast: a bad bean means no listening port."

Creating Your First Project

*Two minutes of setup decisions that you will live with for years: **build tool, Java version, starter set**. Everything else can be changed later.*

1 Spring Initializr

Generate from `https://start.spring.io` or straight from your IDE.

- *Choose **Maven** or **Gradle**.*
- *Pick a Spring Boot version — take the latest **GA**, not a snapshot.*
- *Pick the Java LTS your runtime actually has (17 or 21).*
- *Add only the starters you need today.*

Version currency

*Any tutorial pinned to Boot `3.2.x` is now well behind. Check the Initializr default before copying a `pom.xml` from an article — and note Boot 3.x requires **Java 17 minimum**.*

2 Maven or Gradle

	Maven	Gradle
Config	XML (pom.xml)	Groovy / Kotlin DSL
Build speed	Slower	Faster (incremental + cache)
Learning curve	Gentle	Steeper
Flexibility	Convention-bound	Highly programmable
Typical fit	Enterprise, stable estates	Large or multi-module builds

Either is fine

Both are first-class in Boot. Pick what your team already runs in CI — consistency beats micro-optimisation.

3 Generated layout

```
my-app/
├─ src/main/java/
│   └─ com/example/demo/
│       └─ DemoApplication.java
├─ src/main/resources/
│   ├─ application.yml
│   ├─ static/
│   └─ templates/
├─ src/test/java/
└─ pom.xml
```

Package placement

Keep `DemoApplication` in the root package. Component scanning starts from its package downwards — beans in sibling packages are silently ignored.

4 Minimal pom.xml

```xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <!-- use the current GA release -->
  <version>3.5.x</version>
</parent>

<properties>
  <java.version>21</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

Why no version tags?

The parent POM supplies them. Overriding a single starter version by hand is the fastest way to break a working dependency set.

5 Running it

```
mvn spring-boot:run          # dev loop
mvn clean package            # build the jar
java -jar target/my-app.jar  # run it anywhere
```

Defaults to `http://localhost:8080`. Change with `server.port`, or set it to `0` for a random free port — useful in tests.

6 Setup checklist

- ✓ *Latest GA Boot version, Java 17 or 21.*
- ✓ *Main class in the root package.*
- ✓ *Only the starters you actually use.*
- ✓ *`application.yml` over `.properties` for anything nested.*
- ✓ *A `dev` profile committed, secrets kept out of the repo.*

Interview answer

Be ready to describe the flow end to end: Initializr → starters → `@SpringBootApplication` → auto-config → embedded server → executable jar.

Dependency Injection & IoC

*Inversion of Control means **your classes stop constructing their own collaborators**. The container builds the object graph; your code just declares what it needs.*

1 The container

The Spring IoC container creates, configures, wires and destroys beans. `ApplicationContext` is the interface you actually meet; `BeanFactory` is the lower-level one beneath it.

A bean is just an object

Nothing special about the class. A bean is any object whose lifecycle the container owns — declared via a stereotype annotation, a `@Bean` method, or configuration.

2 Three ways to inject

Style	Verdict
Constructor	**Recommended** — immutable, testable, fails fast
Setter	Acceptable for genuinely optional collaborators
Field	Avoid — hides dependencies, blocks final, hard to test

Correction worth knowing

Since Spring 4.3 `@Autowired` is **optional** on a class with a single constructor. Most tutorials still show it — it is redundant noise.

3 Constructor injection, done properly

```
@Service
public class UserService {

  private final UserRepository repository;
  private final EmailSender emailSender;

  // no @Autowired needed: single constructor
  public UserService(UserRepository repository,
                     EmailSender emailSender) {
    this.repository  = repository;
    this.emailSender = emailSender;
  }
}
```

Why final matters

`final` fields cannot be reassigned or left null. The compiler enforces what would otherwise be a runtime `NullPointerException`.

4 Bean scopes

Scope	One instance per
singleton	Container **(default)**
prototype	Request for the bean
request	HTTP request
session	HTTP session
application	ServletContext
websocket	WebSocket session

Singleton ≠ thread-safe

One instance shared across all threads. Keep mutable state out of your beans — this is the single most common source of production race conditions.

5 Bean lifecycle

Definition read → Instantiate → Populate dependencies → Aware callbacks → @PostConstruct → In use → @PreDestroy

Spelling that trips people up

The annotations are `@PostConstruct` and `@PreDestroy` — not @PreDestrory. In Boot 3.x they come from `jakarta.annotation`, not `javax.annotation`.

6 Why the container is worth it

- Loose coupling — depend on interfaces, not constructors.
- Testability — pass a stub in a unit test, no framework needed.
- Centralised lifecycle — one place decides how objects are built.
- Cross-cutting concerns — transactions, caching and security wrap beans transparently.

Interview answers

What is IoC? The container, not your code, owns object creation and wiring.

Why constructor injection? Dependencies are explicit and mandatory, fields can be final, and the class is usable without Spring at all — which is exactly what makes it unit-testable.

Core Annotations

*Annotations are how you talk to the container. Roughly a dozen carry **most of the weight** in a typical service — learn these properly before chasing the long tail.*

1 @SpringBootApplication

The entry point. It is a convenience annotation combining three others:

- `@Configuration` — *this class can declare beans.*
- `@EnableAutoConfiguration` — *turn on conditional auto-config.*
- `@ComponentScan` — *scan this package and below.*

```
@SpringBootApplication
public class DemoApplication {
  public static void main(String[] args) {
    SpringApplication.run(
      DemoApplication.class, args);
  }
}
```

2 Stereotypes

Annotation	Marks
@Component	Any container-managed bean
@Service	Business logic
@Repository	Data access — plus exception translation
@Controller	MVC controller returning view names
@RestController	@Controller + @ResponseBody
@Configuration	Class declaring @Bean methods

@Repository earns its keep

It is not just a label. It translates vendor-specific persistence exceptions into Spring's `DataAccessException` hierarchy, so your service layer never imports a Hibernate exception type.

3 Wiring and configuration

Annotation	Purpose	Note
@Autowired	Inject a dependency	Optional on a single constructor
@Qualifier	Disambiguate candidates	Use when two beans match
@Primary	Default among candidates	Cleaner than @Qualifier everywhere
@Value	Inject one property	Good for a handful of values
@Bean	Declare a bean manually	For types you do not own
@Profile	Restrict to environments	e.g. dev-only stubs

4 Resolving ambiguity

```
@Service
@Primary
class SmsSender implements Notifier {}

@Service("email")
class EmailSender implements Notifier {}

// picks EmailSender explicitly
public Svc(@Qualifier("email")
           Notifier n) { ... }
```

5 Web layer

Annotation	Binds
@RequestMapping	Any method, class or method level
@GetMapping	HTTP GET
@PostMapping	HTTP POST
@PutMapping / @PatchMapping	Full / partial update
@DeleteMapping	HTTP DELETE
@PathVariable	A segment of the URI
@RequestParam	A query-string parameter
@RequestBody	The deserialised request body

6 Habits worth forming

- *Use the **specific** stereotype, not bare `@Component` — it documents intent.*
- *Prefer constructor injection; drop the redundant `@Autowired`.*
- *Group related properties with `@ConfigurationProperties` instead of scattering `@Value`.*
- *Keep `@Transactional` on the service layer, never on controllers.*

Interview answer

"Stereotypes are all `@Component` underneath — the container treats them identically. The difference is readability, plus the exception translation `@Repository` adds."

Configuration & Profiles

*The same jar must run on a laptop, in CI and in production without a rebuild. That is what **externalised configuration** buys you.*

1 properties vs YAML

```
# application.properties
server.port=8080
spring.datasource.url=jdbc:postgresql://db:5432/app
spring.jpa.hibernate.ddl-auto=validate
```

```
# application.yml
server:
  port: 8080
spring:
  datasource:
    url: jdbc:postgresql://db:5432/app
  jpa:
    hibernate:
      ddl-auto: validate
```

Same keys, different shape

YAML wins once nesting appears. Never keep both files for the same profile — the resolution order will surprise someone.

2 @Value vs @ConfigurationProperties

```
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
  private String name;
  private Duration timeout;
  // getters + setters
}
```

	@Value	@ConfigurationProperties
Best for	One or two values	A group of related keys
Type safety	String parsing	Bound and converted
Validation	Manual	Works with @Validated
Relaxed names	No	Yes

3 Profiles

Profile	File
dev	application-dev.yml
test	application-test.yml
prod	application-prod.yml

```
# activate
--spring.profiles.active=prod
SPRING_PROFILES_ACTIVE=prod
```

Profile-specific wins

`application-prod.yml` overrides `application.yml` key by key — it does not replace the file.

4 Precedence - highest first

#	Source
1	Command-line arguments
2	OS environment variables
3	Profile-specific files (application-{profile}.yml)
4	Base application.yml / .properties
5	Defaults in @Value or the code itself

Why this order matters in containers

It is what lets one immutable image behave differently per environment. The image never changes; the environment variables do.

5 Environment variable mapping

Property	Environment variable
server.port	SERVER_PORT
spring.datasource.url	SPRING_DATASOURCE_URL
app.retry-count	APP_RETRYCOUNT

*Uppercase, dots to underscores. This **relaxed binding** is why Kubernetes and Docker configuration works without any adapter code.*

6 Configuration rules

- ✓ Secrets from environment or a vault — never committed.
- ✓ `ddl-auto: validate` in production; never `update`.
- ✓ Group keys under a prefix and bind them as a type.
- ✓ Fail fast on missing required config — add `@Validated`.
- ✓ Keep `application.yml` as the shared baseline only.

Production hazard

`ddl-auto: update` lets Hibernate alter live schema on startup. It silently drifts from your migrations. Use Flyway or Liquibase and set `validate`.

Building REST APIs

*REST is a set of constraints, not a library. Boot makes the mechanics trivial — the **discipline of resource design** is still yours.*

1 The constraints

- **Client–server** — separate concerns, evolve independently.
- **Stateless** — every request carries everything needed to serve it.
- **Cacheable** — responses declare whether they may be cached.
- **Uniform interface** — consistent resources, verbs and representations.
- **Layered system** — the client cannot tell how many hops sit behind.
- **Code on demand** — optional, and rare in practice.

Stateless in practice

No server-side session affinity. This is precisely what makes horizontal scaling and rolling deploys safe.

2 Methods and semantics

Method	Purpose	Body	Idempotent
GET	Read a resource	No	Yes
POST	Create / non-idempotent action	Yes	No
PUT	Replace a resource wholly	Yes	Yes
PATCH	Update part of a resource	Yes	No*
DELETE	Remove a resource	No	Yes

Idempotent — the definition and the spelling

Idempotent (not inempotent): making the same request many times leaves the server in the same state as making it once. *PATCH can be idempotent, but only if you design the patch document that way.

3 A controller

```
@RestController
@RequestMapping("/api/v1/users")
public class UserController {

  private final UserService service;

  public UserController(UserService service) {
    this.service = service;
  }

  @GetMapping
  Page<UserDto> list(Pageable pageable) {
    return service.list(pageable);
  }

  @PostMapping
  @ResponseStatus(HttpStatus.CREATED)
  UserDto create(@Valid @RequestBody CreateUser req) {
    return service.create(req);
  }
}
```

4 Request path

Client → Controller → Service → Repository

The response travels back the same way, serialised to JSON by Jackson.

Keep controllers thin

A controller should bind input, delegate, and shape the response. The moment business rules appear in one, they become untestable without the web layer.

5 URI design

	Do	Do not
Nouns	/api/users	/api/getUsers
Nesting	/api/users/{id}/orders	/api/userOrders?id=
Versioning	/api/v1/users	unversioned public API
Plurality	consistently plural	mixed /user and /users

6 Design checklist

- ✓ Nouns for resources, HTTP verbs for actions.
- ✓ Correct status codes — never 200 for a failure.
- ✓ DTOs at the boundary; never expose entities.
- ✓ Validate every inbound payload with `@Valid`.
- ✓ Paginate any collection that can grow.

Interview answer

"`@RestController` is `@Controller` plus `@ResponseBody` — return values are serialised to the response body instead of resolved as view names."

Handling Requests

*Four places data arrives from: the **path**, the **query string**, the **body**, and the **headers**. One annotation each — the skill is choosing correctly.*

1 @PathVariable

Identifies which resource. Part of the URI itself.

```
@GetMapping("/users/{id}")
UserDto byId(@PathVariable Long id) {
  return service.find(id);
}
// GET /users/101  ->  id = 101
```

2 @RequestParam

Filters, sorts or pages a collection. Optional by nature.

```
@GetMapping("/users")
List<UserDto> search(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(required = false) String q) {
  return service.search(q, page);
}
// GET /users?q=ali&page=0
```

3 @RequestBody

Carries the payload for a create or update. Deserialised by Jackson.

```
@PostMapping("/users")
@ResponseStatus(HttpStatus.CREATED)
UserDto create(@Valid @RequestBody CreateUserRequest req) {
  return service.create(req);
}
```

```
// request payload
{
  "name":  "Layla Hassan",
  "email": "layla@example.com",
  "age":   28
}
```

4 @RequestHeader

```
@GetMapping("/me")
String me(
  @RequestHeader("Authorization")
  String token) {
  return service.resolve(token);
}
```

Common headers: `Authorization`, `Content-Type`, `Accept`, `X-Request-Id`.

Prefer the filter chain

Reading `Authorization` in a controller means auth logic is scattered. Handle it once in Spring Security.

5 Combining all four

```
@PutMapping("/users/{id}")
UserDto update(@PathVariable Long id,
               @Valid @RequestBody UpdateUser body,
               @RequestParam(defaultValue = "false") boolean notify,
               @RequestHeader("X-Request-Id") String requestId) {
  return service.update(id, body, notify, requestId);
}
// PUT /users/101?notify=true
```

6 Quick reference

Annotation	Reads from	Example
@PathVariable	URI segment	/users/{id}
@RequestParam	Query string	?page=0&size=20
@RequestBody	Request body	JSON payload
@RequestHeader	HTTP header	Authorization
@CookieValue	Cookie	session tracking
@ModelAttribute	Form data	HTML form submit

Interview answer

@RequestBody or @RequestParam?

`@RequestParam` for simple scalar values in the URL. `@RequestBody` for structured payloads.

***The classic mistake:** sending a body with GET. Some clients and proxies drop it silently — if you need a complex query, use POST for a search endpoint.*